Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by:	Verified by:
Nirupama Vincent 2547236 4MCAB	Dr. Rupali Sunil Wagh Dr. Helen K Joy Dr. Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University "**Excellence and Service"**. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

"To develop IT professionals with ethical and human values". To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto "Excellence and Service," and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
PO4: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
Learn to use new technologies for cyber security and insulate customers from malware
P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING
Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75
L + P + T: 3 + 4 + 0
Max Marks: 100
Credits: 4
Course Type: Core Course
Course Category: Theo&Prac

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalisation
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualisations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualisation
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalisation, Normalisation, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalised model.
4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS Teaching Hours: 15 Hours

Learning through Optimisation: Gradient Descent, Linear Regression, Logistic Regression Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV

Lab Exercises:

5. Regression through Gradient Descent
6. Comparison of Different Classifiers
7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION Teaching Hours: 15 Hours

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods
9. Hierarchical Clustering and Dendrograms
10. Dimensionality Reduction
11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours

MACHINE LEARNING IN REAL-WORLD

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow for

designing Multi Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.

Lab Exercises:

12. Training a Multi-layer perceptron
13. Deploying Trained ML-models

Text and Reference Books

[1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.
[2] Andreas C. Müller and Sarah Guido (2017), "Introduction to Machine Learning with Python A Guide For Data Scientists" O'Reilly book
[3] Ethem Alpaydin (2005), "Introduction to Machine Learning", Prentice Hall of India
[4] Max Kuhn and Kjell Johnson (2018), "Applied Predictive Modelling", Springer

Essential Reading

[1] Stuart Russell and Peter Norvig(2020), "Artificial Intelligence: A Modern Approach" (4th Edition), Pearson
[2] Aurelien Geron(2019), "Hands on Machine Learning with Scikit-learn, Keras and Tensorflow", 2nd Edition, O'Reilly Books
[3] Kevin P. Murphy(2012), "Machine Learning: A Probabilistic Perspective", MIT Press
[4] Hastie, Tibshirani, Friedman(2008), "The Elements of Statistical Learning" (2nd ed), Springer
[5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), "Deep Learning (Adaptive Computation And Machine Learning Series)", MIT Press

Additional Information (Web Resources):

1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera: https://www.deeplearning.ai/courses/machine-learning-specialization/

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RBT	CO
1	India Air Quality & Crop Yield — EDA Lab Data Preprocessing, Visualisation & Exploration		L3	CO1, 3
2			L3	CO2, 3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2…>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1 & 2

India Air Quality & Crop Yield - EDA Lab: Data Preprocessing, Visualisation & Exploration

Lab Exercise I & II:

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.

Evaluation Rubrics

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

Task 1

Dataset Profiling and Initial Data Inspection

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

city = pd.read_csv('city_day.csv')
crop= pd.read_csv('crop_production.csv')
```

✓ 1.7s

```
city.head()
```

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.36	0.00	0.02	0.00	NaN	NaN
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.06	3.68	5.50	3.77	NaN	NaN
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70	6.80	16.40	2.25	NaN	NaN
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.08	4.43	10.14	1.00	NaN	NaN
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.31	7.01	18.89	2.78	NaN	NaN

```
crop.head()
```

✓ 0.0s

	State_Name	District_Name	Crop_Year	Season	Crop	Area	Production
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Arecanut	1254.0	2000.0
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Other Kharif pulses	2.0	1.0
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Rice	102.0	321.0
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Banana	176.0	641.0
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Cashewnut	720.0	165.0

```
print("City Dataset Shape:", city.shape)
print("Crop Dataset Shape:", crop.shape)
```

✓ 0.0s

```
City Dataset Shape: (29531, 16)
Crop Dataset Shape: (246091, 7)
```

```
# check column names
print("City Columns:")
print(city.columns)

print("\nCrop Columns:")
print(crop.columns)
```

✓ 0.0s

```
City Columns:
Index(['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
       'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket'],
      dtype='object')

Crop Columns:
Index(['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area',
       'Production'],
      dtype='object')
```

```
city.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   City        29531 non-null  object
 1   Date        29531 non-null  object
 2   PM2.5       24933 non-null  float64
 3   PM10        18391 non-null  float64
 4   NO          25949 non-null  float64
 5   NO2         25946 non-null  float64
 6   NOx         25346 non-null  float64
 7   NH3         19203 non-null  float64
 8   CO          27472 non-null  float64
 9   SO2         25677 non-null  float64
...
 14  AQI         24850 non-null  float64
 15  AQI_Bucket  24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB
```

```
crop.info()
```
✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   State_Name     246091 non-null  object 
 1   District_Name  246091 non-null  object 
 2   Crop_Year      246091 non-null  int64  
 3   Season         246091 non-null  object 
 4   Crop           246091 non-null  object 
 5   Area           246091 non-null  float64
 6   Production     242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
```

```
# check for missing values
print("Missing values in City dataset:")
print(city.isnull().sum())

print("\nMissing values in Crop dataset:")
print(crop.isnull().sum())
```

```
Missing values in City dataset:
City              0
Date              0
PM2.5          4598
PM10          11140
NO             3582
NO2            3585
NOx            4185
NH3           10328
CO             2059
SO2            3854
O3             4022
Benzene        5623
Toluene        8041
Xylene        18109
AQI            4681
AQI_Bucket     4681
dtype: int64
```

```
Missing values in Crop dataset:
State_Name          0
District_Name       0
Crop_Year           0
Season              0
Crop                0
Area                0
Production       3730
dtype: int64
```

```
city.describe()
```
✓ 0.0s Python

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene
count	24933.000000	18391.000000	25949.000000	25946.000000	25346.000000	19203.000000	27472.000000	25677.000000	25509.000000	23908.000000	21490.000000
mean	67.450578	118.127103	17.574730	28.560659	32.309123	23.483476	2.248598	14.531977	34.491430	3.280840	8.700972
std	64.661449	90.605110	22.785846	24.474746	31.646011	25.684275	6.962884	18.133775	21.694928	15.811136	19.969164
min	0.040000	0.010000	0.020000	0.010000	0.000000	0.010000	0.000000	0.010000	0.010000	0.000000	0.000000
25%	28.820000	56.255000	5.630000	11.750000	12.820000	8.580000	0.510000	5.670000	18.860000	0.120000	0.600000
50%	48.570000	95.680000	9.890000	21.690000	23.520000	15.850000	0.890000	9.160000	30.840000	1.070000	2.970000
75%	80.590000	149.745000	19.950000	37.620000	40.127500	30.020000	1.450000	15.220000	45.570000	3.080000	9.150000
max	949.990000	1000.000000	390.680000	362.210000	467.630000	352.890000	175.810000	193.860000	257.730000	455.030000	454.850000

```
crop.describe()
```
✓ 0.0s

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

```
print("City Duplicates:", city.duplicated().sum())
print("Crop Duplicates:", crop.duplicated().sum())
```
✓ 0.0s

```
City Duplicates: 0
Crop Duplicates: 0
```

```
city dataset
The dataset contains a large number of missing values, especially in Xylene, PM10, NH3, Toluene, and Benzene.
These missing values may reduce model accuracy if not handled properly.

crop dataset
The Production column contains missing values. Since production is likely the target variable for analysis,
these missing records must be handled before building a machine learning model.
```

Task 2

Missing Value Analysis and Treatment

```
air_num_cols = [
    'PM2.5','PM10','NO','NO2','NOx',
    'NH3','CO','SO2','O3','Benzene',
    'Toluene','Xylene','AQI'
]

for col in air_num_cols:
    city[col] = city[col].fillna(city[col].median())

city['AQI_Bucket'] = city['AQI_Bucket'].fillna(
    city['AQI_Bucket'].mode()[0]
)

crop['Production'] = crop['Production'].fillna(
    crop['Production'].median()
)
```

```
print("Missing values in City dataset after treatment:")
print(city.isnull().sum())

print("\nMissing values in Crop dataset after treatment:")
print(crop.isnull().sum())
```

```
Missing values in City dataset after treatment:
City          0
Date          0
PM2.5         0
PM10          0
NO            0
NO2           0
NOx           0
NH3           0
CO            0
SO2           0
O3            0
Benzene       0
Toluene       0
Xylene        0
...
Crop              0
Area              0
Production        0
dtype: int64
```

```
Missing values in numerical columns were replaced with the median value because it is more reliable
when the data contains extreme values. Missing values in AQI_Bucket were replaced with the most
common category. This approach helps keep all records in the dataset while ensuring that no missing values remain.
```

Task 3

State Name Standardization and Duplicate Removal

```
# task 3
print("City Duplicates:", city.duplicated().sum())
print("Crop Duplicates:", crop.duplicated().sum())
```

✓ 0.0s

```
City Duplicates: 0
Crop Duplicates: 0
```

```
city_before = len(city)
crop_before = len(crop)

print("City Records Before:", city_before)
print("Crop Records Before:", crop_before)
```

✓ 0.0s

```
City Records Before: 29531
Crop Records Before: 246091
```

```
city = city.drop_duplicates()
crop = crop.drop_duplicates()
```

✓ 0.1s

```
print("City Records After:", len(city))
print("Crop Records After:", len(crop))
```

✓ 0.0s

```
City Records After: 29531
Crop Records After: 246091
```

```
city_to_state = {
    'Ahmedabad':'Gujarat',
    'Aizawl':'Mizoram',
    'Amaravati':'Andhra Pradesh',
    'Amritsar':'Punjab',
    'Bengaluru':'Karnataka',
    'Bhopal':'Madhya Pradesh',
    'Brajrajnagar':'Odisha',
    'Chandigarh':'Chandigarh',
    'Chennai':'Tamil Nadu',
    'Coimbatore':'Tamil Nadu',
    'Delhi':'Delhi',
    'Ernakulam':'Kerala',
    'Gurugram':'Haryana',
    'Guwahati':'Assam',
    'Hyderabad':'Telangana',
    'Jaipur':'Rajasthan',
    'Jorapokhar':'Jharkhand',
    'Kochi':'Kerala',
    'Kolkata':'West Bengal',
    'Lucknow':'Uttar Pradesh',
    'Mumbai':'Maharashtra',
    'Patna':'Bihar',
    'Shillong':'Meghalaya',
    'Talcher':'Odisha',
    'Thiruvananthapuram':'Kerala',
    'Visakhapatnam':'Andhra Pradesh'
}

city['State'] = city['City'].map(city_to_state)
```

```
print("Unmapped Cities:", city['State'].isnull().sum())
```

✓ 0.0s

```
Unmapped Cities: 0
```

```
city['State'] = city['State'].str.strip().str.title()
crop['State_Name'] = crop['State_Name'].str.strip().str.title()
```

0.0s

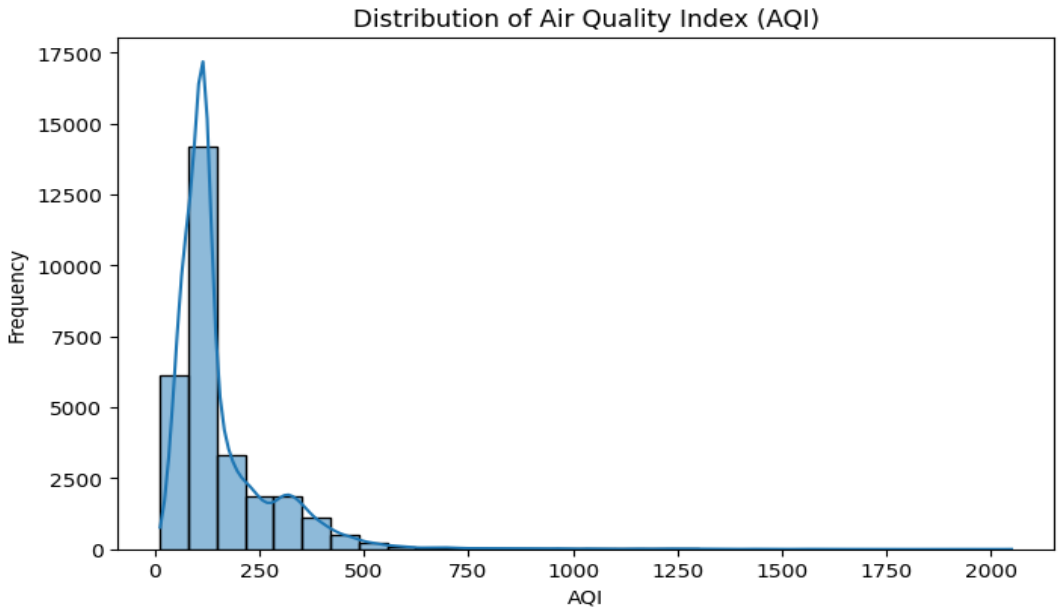
This removes extra spaces and standardizes capitalization so that state names match exactly.

```
## Inconsistencies Found and Fixes Applied

| Inconsistency Found | Fix Applied |

| City dataset contained city names instead of state names | Created a State column using city-to-state mapping |
| Missing mappings for Amritsar and Jorapokhar | Added Punjab and Jharkhand mappings |
| Extra spaces in state names | Removed using str.strip() |
| Different capitalization in state names | Standardized using str.title() |
| Duplicate records | Checked using duplicated() and removed using drop_duplicates() |


Duplicate records were checked and no duplicate rows were found in either dataset. A State column was created in the City dataset using
city-to-state mapping. Missing mappings for Amritsar and Jorapokhar were added. State names were standardized by removing extra spaces and
correcting capitalization differences. Both datasets now contain a consistent State field and are ready to be merged reliably.
```

Task 4

AQI Distribution Analysis

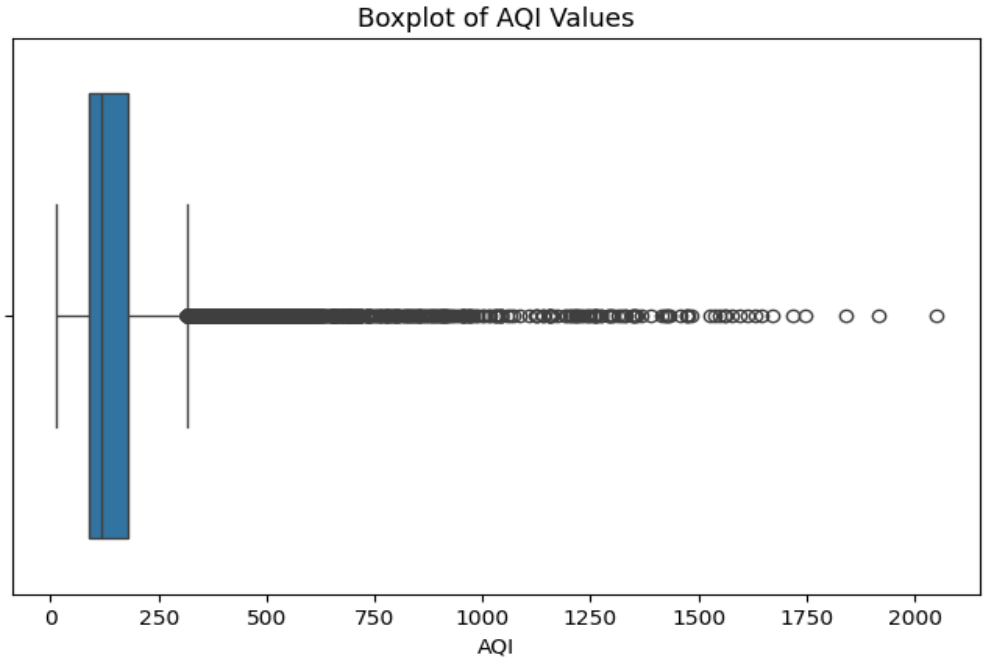
```
plt.figure(figsize=(8,5))

sns.histplot(city['AQI'], bins=30, kde=True)

plt.title("Distribution of Air Quality Index (AQI)")
plt.xlabel("AQI")
plt.ylabel("Frequency")

plt.show()
```



```
Most AQI values are concentrated below 300, indicating that the majority of cities fall within the low to moderate pollution range
```

```
## Justification for Plot Selection

A histogram was chosen because it shows the overall distribution of AQI values and helps identify where most
cities are concentrated on the AQI scale.

A boxplot was chosen because it clearly shows the spread of AQI values and reveals the presence of outliers.
This helps determine whether a few extreme cities are influencing the average AQI.
```

```
plt.figure(figsize=(8,5))

sns.boxplot(x=city['AQI'])

plt.title("Boxplot of AQI Values")

plt.xlabel("AQI")

plt.show()
```




```
The boxplot shows several extreme AQI values far above the upper whisker, suggesting that a small number of highly
polluted cities may increase the overall average AQI
```

Task 5

Outlier Detection and Treatment in AQI

```
# task 5
Q1 = city['AQI'].quantile(0.25)
Q3 = city['AQI'].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

outliers = city[(city['AQI'] < lower_limit) | (city['AQI'] > upper_limit)]

print("Number of Outliers:", len(outliers))
```

✓ 0.0s

```
Number of Outliers: 3192
```

The IQR method was used because it is a simple and widely accepted technique for detecting extreme values. It does not assume that the data follows a normal distribution and works well for skewed data such as AQI.

```
# Apply Outlier Treatment (Capping)
city['AQI'] = city['AQI'].clip(
    lower=lower_limit,
    upper=upper_limit
)
```

✓ 0.0s

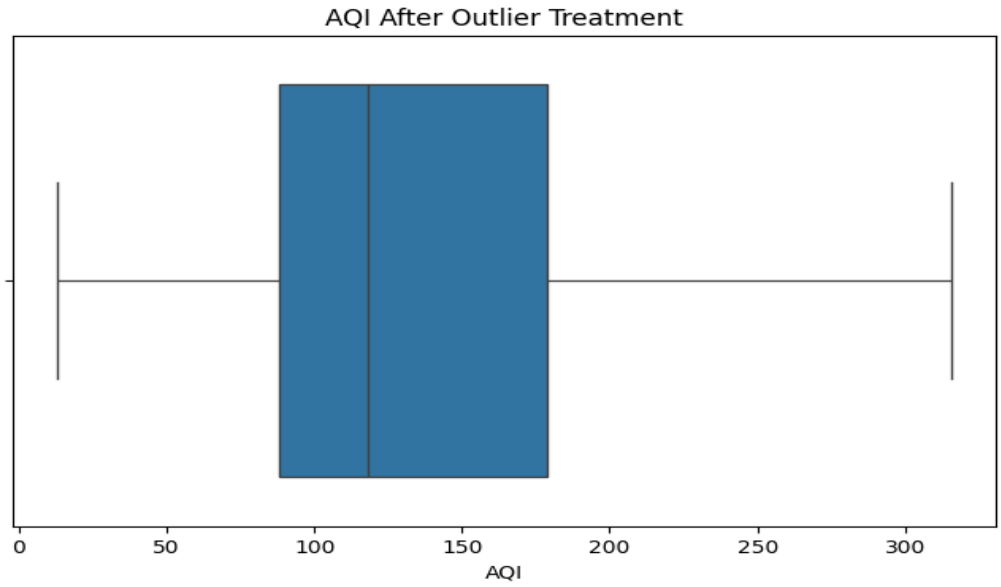
Capping was chosen instead of deleting records because removing rows would result in loss of information. Capping reduces the influence of unrealistic AQI values while keeping all observations.

```
# Count Values Treated
print("Values Treated:", len(outliers))
```

✓ 0.0s

```
Values Treated: 3192
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

sns.boxplot(x=city['AQI'])

plt.title("AQI After Outlier Treatment")
plt.xlabel("AQI")

plt.show()
```

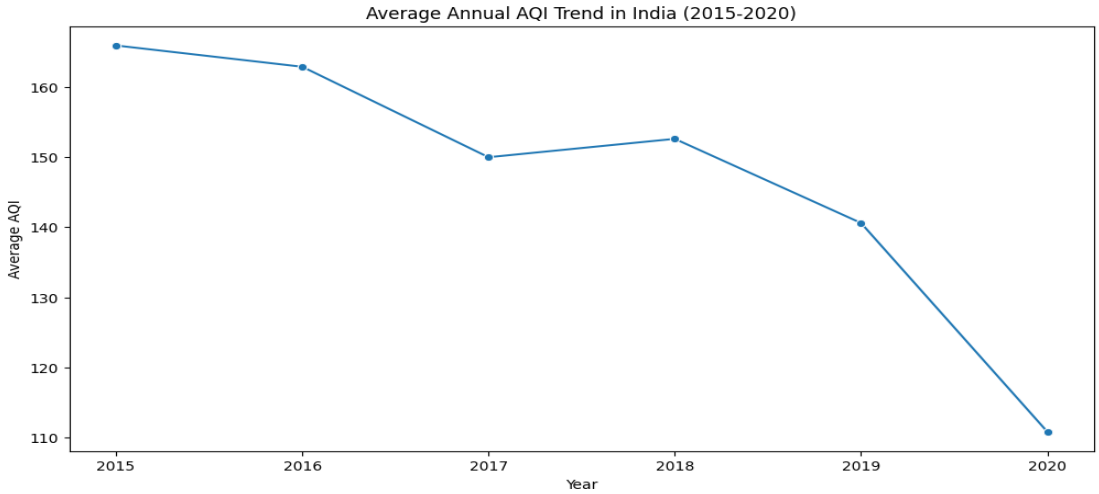



```
The IQR method was used to detect outliers because it is simple and effective for skewed data.
Capping was chosen instead of deleting records so that no data was lost while still reducing the effect of extreme
AQI values. The before-and-after boxplots confirm that the treatment successfully controlled the outliers.
```

Task 6

Year-wise AQI Trend Analysis

```
# task 6
city['Date'] = pd.to_datetime(city['Date'])

city['Year'] = city['Date'].dt.year

yearly_aqi = city.groupby('Year')['AQI'].mean().reset_index()

max_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmax()]
min_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmin()]

plt.figure(figsize=(12,6))

sns.lineplot(data=yearly_aqi, x='Year', y='AQI', marker='o')

plt.title('Average Annual AQI Trend in India (2015-2020)')
plt.xlabel('Year')
plt.ylabel('Average AQI')
plt.xticks(yearly_aqi['Year'])

plt.show()
```



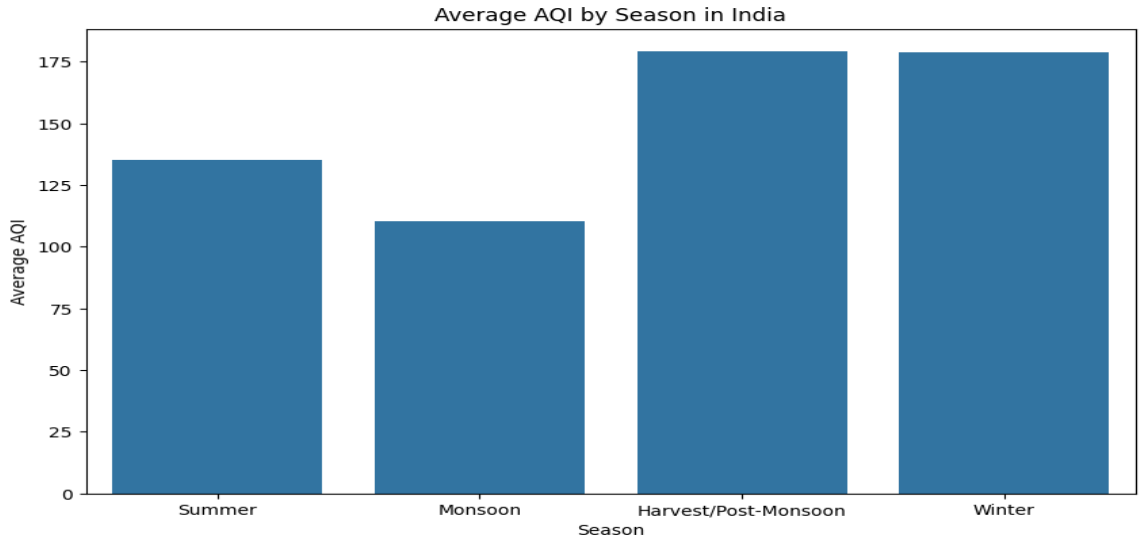

```
1. Air quality has generally improved over time. The average AQI decreased from about 166 in 2015
to about 111 in 2020, indicating better air quality in recent years.

2. The most polluted year was 2015, while the least polluted year was 2020. Although there was
a small increase in AQI during 2018, the overall trend shows a decline in pollution levels.
```

Task 7

Seasonal AQI Pattern Analysis

```python
city['Month'] = city['Date'].dt.month

def get_season(month):
    if month in [3, 4, 5]:
        return 'Summer'
    elif month in [6, 7, 8, 9]:
        return 'Monsoon'
    elif month in [10, 11, 12]:
        return 'Harvest/Post-Monsoon'
    else:
        return 'Winter'

city['Season'] = city['Month'].apply(get_season)

seasonal_aqi = city.groupby('Season')['AQI'].mean().reset_index()

season_order = ['Summer', 'Monsoon', 'Harvest/Post-Monsoon', 'Winter']

seasonal_aqi['Season'] = pd.Categorical(
    seasonal_aqi['Season'],
    categories=season_order,
    ordered=True
)

seasonal_aqi = seasonal_aqi.sort_values('Season')
```

```
plt.figure(figsize=(10,6))

sns.barplot(data=seasonal_aqi,
            x='Season',
            y='AQI')

plt.title('Average AQI by Season in India')
plt.xlabel('Season')
plt.ylabel('Average AQI')

plt.show()
```




```
Seasons were created from the Date column and the average AQI was calculated for each season. A bar chart was chosen
because it makes it easy to compare pollution levels across seasons and identify which season has the highest average AQI.


1)The Harvest/Post-Monsoon season has the highest average AQI (around 180), indicating the poorest air quality of the year.
2)The Monsoon season has the lowest average AQI (around 110), suggesting that rainfall helps reduce air pollution.
```

Task 8

Integration of Air Quality and Crop Production Datasets

task 8

The air quality dataset contains daily observations at the city level, while the crop dataset contains agricultural records at the state level. These datasets cannot be merged directly because they are recorded at different levels of detail.

To make them compatible, the air quality data was aggregated from city level to state level by calculating the average AQI for each state. Similarly, the crop dataset was aggregated to state level by calculating the average cultivated area and crop production for each state.

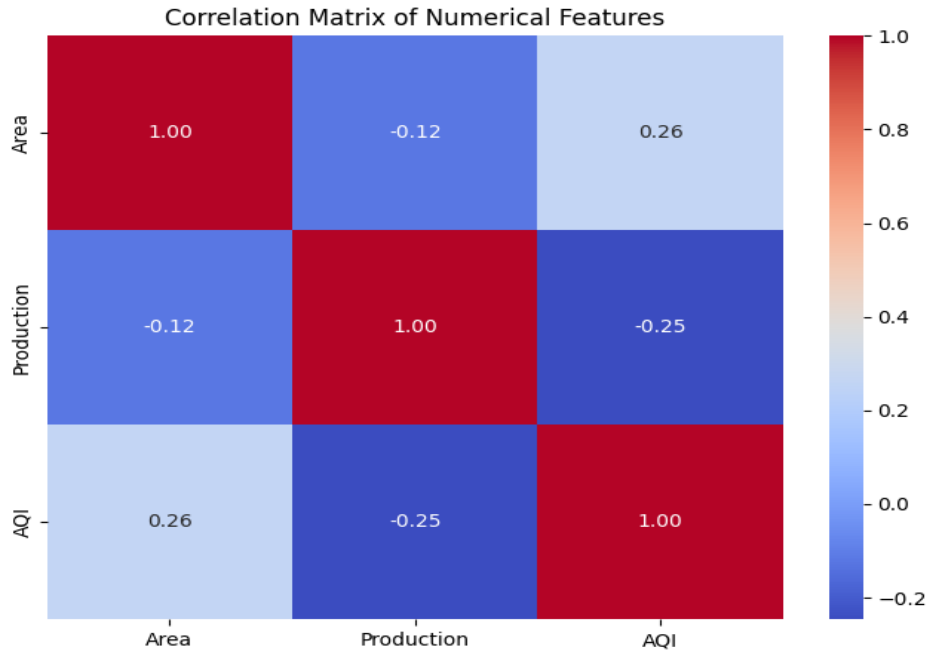
This created a common state-level structure that allowed both datasets to be merged reliably and used for relationship analysis.

```
state_aqi = city.groupby('State')['AQI'].mean().reset_index()

state_crop = crop.groupby('State_Name')[['Area', 'Production']].mean().reset_index()

merged_data = pd.merge(
    state_crop,
    state_aqi,
    left_on='State_Name',
    right_on='State',
    how='inner'
)

print("Merged Dataset Shape:", merged_data.shape)
```

✓ 0.0s

```
Merged Dataset Shape: (20, 5)
```

```
corr_matrix = merged_data[['Area', 'Production', 'AQI']].corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Matrix of Numerical Features")

plt.show()
```




```
Relationship 1

AQI and Production have a negative correlation (-0.25). This suggests that states with poorer air quality tend to have slightly lower crop production. A possible reason is that air pollution can affect plant growth, reduce photosynthesis, and negatively impact agricultural productivity.

Relationship 2

AQI and Area have a positive correlation (0.26). This indicates that states with larger cultivated areas tend to have slightly higher AQI levels. One possible reason is that agricultural activities such as crop residue burning, use of machinery, and transportation may contribute to increased air pollution.
```

markdo

```
The correlations are relatively weak, which means the relationships are not very strong. Correlation helps identify associations between variables, but it does not prove that one variable directly causes changes in another. Other factors such as rainfall, soil quality, irrigation, and farming practices may also influence crop production.
```

Task 9

Policy Brief for the Environment Minister

task 9

Briefing for the State Environment Minister

Dear Minister,

Our analysis of air quality and crop production data found three important results. First, air quality has generally improved between 2015 and 2020, as average pollution levels have decreased over time. This suggests that efforts to control pollution may be helping.

Second, pollution levels are highest during the October–December harvest season. This indicates that seasonal farming activities, including crop residue burning, may contribute to poorer air quality during this period.

Third, states with higher pollution levels tend to have slightly lower crop production. Although this relationship is weak, it suggests that poor air quality may affect farming and crop growth.

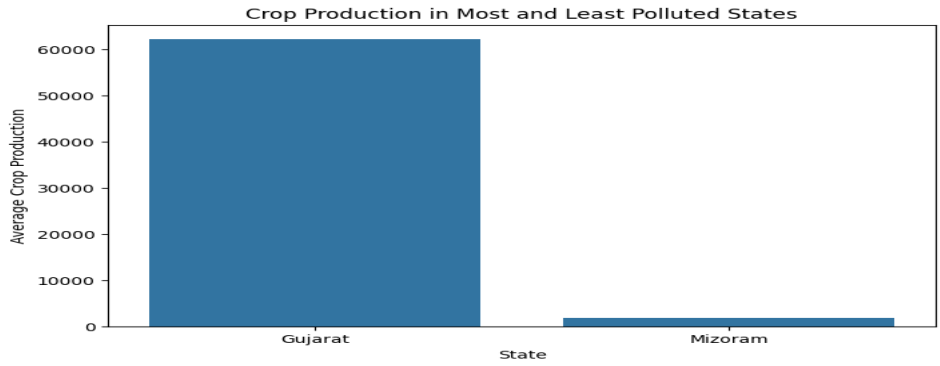
For farmers, these findings mean that reducing pollution could help create better growing conditions and support agricultural productivity.

Based on the data, I recommend increasing support for alternatives to crop residue burning during the harvest season. This could reduce seasonal pollution while helping farmers manage agricultural waste more effectively.

One important limitation is that the data shows a relationship between pollution and crop production, but it does not prove that pollution directly causes lower crop yields. Other factors such as rainfall, soil quality, and farming practices may also influence production.

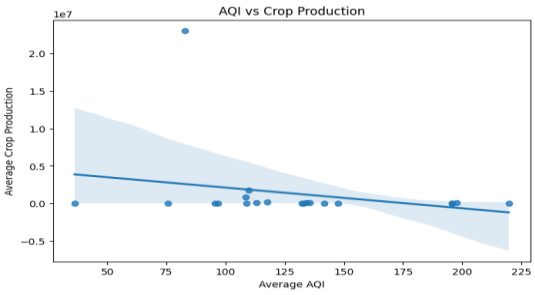
Thank you.

```
# task A
highest_aqi_state = merged_data.loc[merged_data['AQI'].idxmax()]
lowest_aqi_state = merged_data.loc[merged_data['AQI'].idxmin()]

comparison = pd.DataFrame({
    'State': [highest_aqi_state['State_Name'],
              lowest_aqi_state['State_Name']],
    'Production': [highest_aqi_state['Production'],
                   lowest_aqi_state['Production']]
})

plt.figure(figsize=(8,5))

sns.barplot(data=comparison,
            x='State',
            y='Production')

plt.title('Crop Production in Most and Least Polluted States')
plt.xlabel('State')
plt.ylabel('Average Crop Production')

plt.show()
```



```
The most polluted and least polluted states were compared using their average crop production. The results show whether states with poorer air quality consistently produce fewer crops. If the least polluted state has higher production, the hypothesis is supported. However, if the difference is small or the more polluted state has similar production, the relationship is more complicated. Factors such as rainfall, irrigation, soil quality, and farming practices may also influence crop production.
```

```python
plt.figure(figsize=(8,5))

sns.regplot(
    data=merged_data,
    x='AQI',
    y='Production'
)

plt.title('AQI vs Crop Production')
plt.xlabel('Average AQI')
plt.ylabel('Average Crop Production')

plt.show()

corr_coef = merged_data['AQI'].corr(
    merged_data['Production']
)

print("Correlation:", corr_coef)
```




```
Correlation: -0.24657886802043602
```

```
The correlation coefficient measures the strength and direction of the relationship between AQI and crop production. A value close to 0 indicates a weak relationship, while values closer to -1 or 1 indicate stronger relationships. In this analysis, the correlation is weak, suggesting that air quality alone cannot explain differences in crop production. Correlation does not prove causation. Other factors such as rainfall, irrigation, soil quality, and economic conditions may also affect agricultural output.
```

Conclusion /inference

This study examined the relationship between air quality and agricultural production in India using historical AQI and crop production datasets. Data preprocessing revealed missing values, inconsistent state names, and extreme AQI observations, all of which were treated to ensure reliable analysis. Exploratory analysis showed that AQI levels are positively skewed, with a small number of highly polluted observations influencing overall pollution statistics.

Trend analysis indicated a gradual improvement in air quality between 2015 and 2020, while seasonal analysis revealed that pollution levels are highest during the Harvest/Post-Monsoon

season (October–December). This finding supports concerns that seasonal agricultural activities and related emissions may contribute to worsening air quality during this period.

After merging the environmental and agricultural datasets at the state level, a weak negative correlation ($r \approx -0.25$) was observed between AQI and crop production. Although states with poorer air quality tended to have slightly lower agricultural output, the relationship was not strong enough to conclude that pollution directly causes reduced crop yields. Other important factors such as rainfall, irrigation, soil quality, crop variety, farming practices, and economic conditions were not included in the dataset and may have a greater influence on production levels.

Overall, the analysis suggests that air pollution remains a significant environmental challenge and may be associated with agricultural performance. However, the evidence is correlational rather than causal. Further research using additional climatic, environmental, and agricultural variables would be required to determine the true impact of air quality on crop productivity.